Interactive 3-D Graph Renderer Specification

Optional Plotly backend for spatial decorated-graph inspection

mdstats

Contents

1 Purpose and status

This document is the normative specification for implemented Stage G5 in `mdstats.plotting.graph_3d`. It defines an optional Plotly renderer for interactive three-dimensional inspection of decorated graphs.

The API is implemented and validated in `mdstats` 0.11.0.

2 Motive

Dense atomistic and framework graphs are often difficult to interpret after 2-D projection. Interactive 3-D rendering allows the user to rotate the graph, inspect periodic replicas, zoom into local environments, and read scientific metadata through hover text.

The 3-D renderer is intended primarily for diagnosis and exploration. The existing Matplotlib backend remains the preferred route for stable publication figures.

The renderer must consume the same graph-view, style, selection, and periodic-display contracts as the 2-D renderer. It must not become a second graph-analysis system.

3 Normative ownership

This specification owns:

- Plotly-specific rendering options;
- conversion of resolved node and edge styles into 3-D traces;
- interactive hover data;
- unit-cell wireframes;
- camera and scene configuration;
- Plotly trace grouping;
- renderer-specific complexity safeguards;
- HTML export through the render result;
- optional dependency behavior.

It does not own:

- graph identity;
- source focus or filtering semantics;
- periodic replication or local unwrapping;
- scientific metadata construction;
- chemical palette definitions;
- framework, ring, site, or cage analysis;
- trajectory animation or callback applications.

4 AI context summary

1. `plot_decorated_graph_3d()` consumes a `DecoratedGraphView` and delegates periodic materialization to G4.
2. Plotly is imported lazily and remains optional.
3. The returned figure is diagnostic output, not scientific state.
4. Node and edge hover text must preserve stable source identities.
5. Replicas and ghosts remain visibly distinguishable and traceable to source keys.
6. Unit-cell geometry uses the same row-vector cell convention as the scientific data.
7. Edge gradients are approximations made of explicit line segments.
8. No automatic style degradation, object sampling, or image-range reduction is allowed.
9. Unsupported directed arrows, self-loops, or overlapping multiedges must produce an explicit error or warning according to options.
10. The renderer must remain usable without importing Matplotlib internals.

5 Module and dependency policy

The public module is

`mdstats.plotting.graph_3d`

Plotly must be an optional dependency. The package-level import

```
import mdstats
```

must succeed when Plotly is absent.

Calling the 3-D renderer without Plotly installed must raise `GraphOptionalDependencyError` with an installation hint such as

```
pip install mdstats[interactive]
```

The recommended optional dependency is a compatible `plotly` release recorded in `pyproject.toml`.

6 Public rendering options

```python
@dataclass(frozen=True, slots=True)
class Graph3DRenderOptions:
    width: int = 1000
    height: int = 800
    title: str | None = None

    show_axes: bool = False
    equal_aspect: bool = True
    background: Literal["light", "dark", "transparent"] = "light"

    camera_projection: Literal["perspective", "orthographic"] = "orthographic"
    camera_eye: tuple[float, float, float] | None = None
    uirevision: str | None = "mdstats-graph-3d"

    cell_mode: Literal[
        "auto",
        "none",
        "reference",
        "all",
        "outer_boundary",
    ] = "auto"
    cell_color: str = "#666666"
    cell_width: float = 3.2
    cell_alpha: float = 0.72

    show_legend: bool = True
    node_hover: bool = True
    edge_hover: bool = True
    hover_float_precision: int = 4

    edge_color_mode: Literal[
        "constant",
        "midpoint_split",
        "segmented_gradient",
    ] = "midpoint_split"
    gradient_segments: int = 8

    directed_edge_mode: Literal["reject", "line_only"] = "reject"
    allow_parallel_overlap: bool = False
    max_plotly_traces: int = 512
```

7 Option constraints

7.1 Dimensions

`width` and `height` must be positive integers. They set the initial browser or notebook figure size in pixels.

7.2 Background

The renderer must provide documented light, dark, and transparent scene presets. Transparent mode sets both paper and scene backgrounds transparent.

7.3 Camera

`camera_projection` selects Plotly perspective or orthographic projection. Orthographic is the default because it avoids perspective distortion of atomistic cell metrics.

`camera_eye`, when supplied, must contain three finite floats and must not be the zero vector. It is passed as the initial Plotly scene eye position.

`uirevision` preserves the user's camera when nonstructural figure updates occur. A value of `None` disables persistence.

7.4 Cell mode

`cell_mode` controls display of lattice wireframes:

`auto` Draw the reference cell when a finite cell is available; otherwise draw no cell.
`none` Draw no cell.
`reference` Draw the reference cell at image shift `(0, 0, 0)`.
`all` Draw every primary cell recorded by `PeriodicGraphView`.
`outer_boundary` Draw one bounding parallelepiped for a rectangular expanded image range.

`reference`, `all`, and `outer_boundary` require a finite cell. `all` and `outer_boundary` additionally require periodic preparation metadata sufficient to define the requested cells. Invalid combinations are errors.

7.5 Hover precision

`hover_float_precision` must be an integer from 0 through 12. It affects display text only and never modifies stored coordinates or metadata.

7.6 Edge color mode

`constant` Draw each edge with its resolved edge color.
`midpoint_split` Split each edge at its Cartesian midpoint. The first half uses the source-node color and the second half uses the target-node color, while width, alpha, and dash remain controlled by the resolved edge style.
`segmented_gradient` Divide each edge into `gradient_segments` equal intervals and linearly interpolate endpoint colors.

`gradient_segments` must be at least 2. It is ignored by other modes.

7.7 Trace limit

`max_plotly_traces` is a renderer-specific upper bound on the number of Plotly traces after style grouping. It must be positive. Exceeding it raises `GraphComplexityError`; the renderer must not silently merge scientifically distinct style groups.

8 Interactive render result

```python
@dataclass(slots=True)
class InteractiveGraphRenderResult:
    figure: Any
    periodic_view: PeriodicGraphView

    rendered_node_keys: tuple[PeriodicNodeKey, ...]
    rendered_edge_keys: tuple[PeriodicEdgeKey, ...]

    node_trace_indices: Mapping[str, tuple[int, ...]]
    edge_trace_indices: Mapping[str, tuple[int, ...]]
    hover_trace_indices: Mapping[str, tuple[int, ...]]
```

```
    cell_trace_indices: tuple[int, ...]

    complexity: GraphComplexityReport
    style_metadata: Mapping[str, Any]
    render_metadata: Mapping[str, Any]
    warnings: tuple[str, ...]

    def to_html(
        self,
        *,
        include_plotlyjs: Literal["cdn", "directory"] | bool = "cdn",
        full_html: bool = True,
    ) -> str:
        ...

    def write_html(
        self,
        path: str | os.PathLike[str],
        *,
        include_plotlyjs: Literal["cdn", "directory"] | bool = "cdn",
        full_html: bool = True,
        auto_open: bool = False,
    ) -> None:
        ...
```

`figure` is a Plotly `go.Figure`, typed as `Any` so importing the result class does not require Plotly.

Trace-index mappings allow downstream notebook code to identify renderer groups without relying on undocumented insertion order.

The result owns no scientific arrays beyond the immutable `PeriodicGraphView`.

9 Public function

```
def plot_decorated_graph_3d(
    view: DecoratedGraphView,
    *,
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    options: Graph3DRenderOptions | None = None,
) -> InteractiveGraphRenderResult:
    ...
```

`None` selects documented defaults:

- `CanonicalCellDisplay()`;
- `GraphStyle.atomic_default()` only when the caller uses an atomic convenience wrapper, otherwise `GraphStyle.default()`;
- no focus or filter;
- default complexity policy;
- default 3-D render options.

The generic renderer never guesses a scientific adapter preset from metadata.

10 Rendering pipeline

The function must execute these conceptual stages:

```
1. validate public inputs without importing Plotly
2. lazily import Plotly or raise GraphOptionalDependencyError
3. call prepare_periodic_graph_view()
4. validate finite 3-D display coordinates
5. resolve node and edge styles against the display graph
6. estimate line segments, hover points, labels, and trace groups
7. enforce common complexity policy and max_plotly_traces
8. construct grouped node traces unless node display is hidden
9. construct grouped edge traces
10. construct optional edge-hover marker traces
11. construct requested unit-cell wireframes
12. configure scene, camera, aspect, axes, legend, and background
13. return InteractiveGraphRenderResult
```

The renderer must not perform periodic unwrapping or replication itself.

11 Coordinate and aspect handling

Displayed coordinates are taken directly from `PeriodicGraphView.graph.node_positions_3d`.

For equal aspect, the Plotly scene should use data-proportional scaling. If one axis has zero extent, the renderer must apply a small display-only padding rather than altering node coordinates.

Axis labels are Cartesian `x`, `y`, and `z` unless future metadata explicitly supplies units. Axes are hidden by default to reduce clutter.

12 Node traces

Nodes should be grouped by compatible resolved style, including:

- marker symbol;
- fill color;
- size;
- alpha;
- outline color;
- outline width;
- legend label;
- display role when role-specific styling differs.

Plotly marker symbols are more limited than Matplotlib markers. The first renderer must document and support at least:

`o, s, ^, v, d, x, +, *`

Unsupported symbols raise `GraphUnsupportedFeatureError`; they must not be silently changed.

Ghost and replica nodes use the same source metadata but may receive style patches through the display-role attribute.

12.1 Node-display modes

The Plotly backend consumes the same `GraphStyle.node_display_mode` contract as the 2-D backend.

`markers` Group and render the fully resolved node markers.

`dots` Preserve resolved node color and alpha while replacing the marker by a small circle. Plotly receives a marker diameter derived from the shared `node_dot_size` area.

`hidden` Create no node traces. Node hover and node text are therefore unavailable, and node legend entries are absent. The result still contains every prepared periodic node key and source mapping; edge traces, cell wireframes, and edge hover traces are unchanged.

Hidden nodes do not reduce periodic materialization costs because replicas and ghosts are still required to define correct edge geometry. They only reduce browser trace and marker load.

13 Edge traces

13.1 Straight-line geometry

For explicit endpoint positions $\mathbf{x}_a$ and $\mathbf{x}_b$, a straight edge is

$$\mathbf{x}(t) = (1-t)\mathbf{x}_a + t\mathbf{x}_b, \qquad 0 \le t \le 1.$$

The initial G5 renderer uses straight segments. Curved parallel-edge displacement is deferred.

13.2 Compatible style grouping

Edges with identical renderer-compatible style should share a `Scatter3d` line trace using `None` separators. Grouping keys include:

- color or endpoint-color pair;
- width;
- alpha;
- supported dash pattern;
- transition or bridge legend class;
- display role when visually distinct.

Stable edge-key mappings must survive grouping.

13.3 Dash support

The renderer must support Plotly-compatible forms of

`solid, dot, dash, dashdot`

A Matplotlib-specific custom dash tuple is unsupported in the first G5 renderer and raises `GraphUnsupportedFeatureError`.

13.4 Midpoint split

For source color $\mathbf{c}_s$ and target color $\mathbf{c}_t$, the edge midpoint is

$$\mathbf{x}_m = \frac{\mathbf{x}_a + \mathbf{x}_b}{2}.$$

The source half uses $\mathbf{c}_s$, and the target half uses $\mathbf{c}_t$.

13.5 Segmented gradient

For K segments, define

$$t_k = \frac{k}{K}, \qquad \mathbf{x}_k = (1-t_k)\mathbf{x}_a + t_k\mathbf{x}_b,$$

and interpolate color in RGBA space:

$$\mathbf{c}_k = (1-t_k)\mathbf{c}_s + t_k\mathbf{c}_t.$$

This mode can generate many traces or trace groups. The requested primitive count must be included in complexity reporting.

14 Hover data

14.1 Node hover

Node hover should include:

- stable source node key;
- display image shift;
- display role;
- Cartesian position;
- selected source node attributes;
- scientific adapter schema and frame context when available.

The renderer should prioritize common fields such as atom index, symbol, degree, component, ring ID, site ID, or cage ID without requiring them.

14.2 Edge hover

Plotly line hover is unreliable for long grouped line traces. The renderer should add an invisible or nearly invisible marker trace at each edge midpoint when `edge_hover=True`.

Edge hover should include:

- stable source edge key;
- displayed endpoint source keys;
- endpoint image shifts;
- display role;
- Cartesian edge length;
- selected source edge attributes;
- transition status or linker path when available.

Hover markers are display aids and do not enter graph identity or legends.

15 Labels

The shared `GraphLabelOptions` remains authoritative. The first 3-D renderer supports node text labels. Edge labels are hover-only.

Labels are off by default. Requested labels count toward the common complexity policy. If a label attribute is missing, style validation must fail before Plotly trace construction.

16 Unit-cell wireframes

16.1 Reference cell

The eight corners of cell image $\mathbf{q}$ are

$$\mathbf{r}(\mathbf{q}, \mathbf{n}) = (\mathbf{q} + \mathbf{n})H, \qquad \mathbf{n} \in \{0, 1\}^3.$$

The renderer connects the twelve parallelepiped edges.

16.2 All primary cells

For `cell_mode="all"`, draw the cell at every `primary_cell_image_shifts` entry. Duplicate wireframe segments shared by adjacent cells may be deduplicated to reduce overdraw.

16.3 Outer boundary

For rectangular expanded ranges $[l_\alpha, u_\alpha]$, the outer boundary spans fractional corners from

$$(l_1, l_2, l_3)$$

to

$$(u_1 + 1, u_2 + 1, u_3 + 1).$$

This mode is invalid for a nonrectangular future primary-image set.

Cell wireframes are renderer annotations. They are not graph edges and must not appear in rendered scientific edge keys.

17 Legend behavior

Legends should represent meaningful style classes, not every Plotly trace. Multiple trace groups with the same legend label must use one visible legend entry through Plotly legend grouping.

Ghosts and replicas should appear in the legend only when their style differs from canonical nodes or when explicitly requested by a future option.

18 Complexity model

The renderer must report:

- source and selected graph counts from G4;
- display nodes and edges;
- requested node labels;
- edge line primitives;
- gradient segments;
- edge-hover marker count;
- unit-cell line count;
- final Plotly trace count.

For N_E display edges:

```
constant             approximately N_E line segments
midpoint_split        approximately 2 N_E line segments
segmented_gradient    approximately K N_E line segments
```

The common `max_gradient_segments` limit applies to segmented edge primitives. The renderer-specific `max_plotly_traces` applies after grouping.

No automatic fallback from segmented gradient to midpoint or constant color is allowed. The user must request a simpler mode.

19 Directed graphs, multigraphs, and self-loops

19.1 Directed graphs

The first renderer does not draw 3-D arrowheads.

- `directed_edge_mode="reject"` raises `GraphUnsupportedFeatureError` for a directed graph;
- `directed_edge_mode="line_only"` draws directed edges as ordinary lines and records a warning that direction is available only through hover metadata.

19.2 Parallel edges

Straight parallel edges overlap exactly when they share endpoint images.

- `allow_parallel_overlap=False` raises `GraphUnsupportedFeatureError`;
- `True` renders the overlap and records a warning.

Curved parallel-edge separation is deferred.

19.3 Self-loops

Self-loops with coincident displayed endpoints are unsupported and raise `GraphUnsupportedFeatureError`. Periodic self-edges materialized between distinct images may be rendered as ordinary straight edges if G4 supports them.

20 Background and color handling

The renderer consumes colors resolved by `GraphStyle`. It must accept any color format validated by the style module and convert it deterministically to Plotly RGBA strings.

Alpha from node or edge style multiplies the color alpha. The renderer must not alter palette values to suit the selected background. Dark-mode publication tuning belongs to explicit style presets.

21 Export behavior

`InteractiveGraphRenderResult.to_html()` returns Plotly HTML without writing a file. `write_html()` writes the same representation.

Recommended defaults use `include_plotlyjs="cdn"` for compact files. Users requiring a self-contained artifact may pass `True`.

The renderer does not automatically export static PNG, SVG, or PDF through Kaleido. Static 2-D publication output remains owned by the Matplotlib renderer. Future static 3-D export may be added as an optional extension.

22 Error and warning policy

22.1 Errors

Raise an explicit visualization exception for:

- missing Plotly dependency;
- missing or nonfinite 3-D positions;
- invalid render dimensions or camera values;
- incompatible cell mode;
- unsupported marker or dash style;
- directed graph under reject mode;
- overlapping multiedges when disallowed;
- unsupported self-loops;
- invalid label attributes;
- complexity or trace-count overflow;
- inconsistent periodic result mappings;
- HTML path or write failure.

22.2 Warnings

Record warnings for:

- line-only rendering of a directed graph;
- intentionally overlapping parallel edges;

- residual cycle or boundary ghosts;
- hidden axes on a graph without a unit-cell reference;
- nearly coincident nodes;
- very large hover payloads;
- transparent backgrounds that may reduce contrast;
- equal-aspect padding for degenerate extents.

23 Public/private responsibility

Public:

```
Graph3DRenderOptions
InteractiveGraphRenderResult
plot_decorated_graph_3d
```

Private and replaceable:

```
Plotly import helper
style-to-trace grouping
marker-symbol translation
line-dash translation
RGBA interpolation
edge-segment batching
hover-text formatting
edge-hover midpoint trace construction
cell-wireframe generation
camera and scene assembly
trace-count estimation
```

24 Atomic-connectivity convenience API

The generic renderer remains scientific-domain agnostic. The atomic-connectivity adapter provides the following public wrappers:

```python
def plot_atomic_connectivity_3d(
    collection: AtomisticFrameCollection,
    connectivity: AtomicConnectivityState | AtomicConnectivityResult,
    *,
    frame_index: int,
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
    complexity_policy: GraphComplexityPolicy | None = None,
    options: Graph3DRenderOptions | None = None,
) -> InteractiveGraphRenderResult:
    ...

def plot_connectivity_transition_3d(
    collection: AtomisticFrameCollection,
    result: AtomicConnectivityResult,
    *,
    transition_id: int,
    coordinate_frame: Literal["before", "after"] = "after",
    periodic: PeriodicDisplayOptions | None = None,
    style: GraphStyle | None = None,
    focus: GraphFocus | None = None,
    graph_filter: GraphFilter | None = None,
```

```
    complexity_policy: GraphComplexityPolicy | None = None,
    options: Graph3DRenderOptions | None = None,
) -> InteractiveGraphRenderResult:
    ...
```

The atomic adapter owns these wrappers and atomic default styles. The generic 3-D renderer owns only backend behavior.

25 Testing requirements

25.1 Optional dependency tests

- base package imports without Plotly;
- 3-D calls without Plotly raise the documented optional-dependency error;
- interactive extra installs and imports successfully.

25.2 Options and result tests

- invalid dimensions, camera, precision, and trace limits fail;
- result mappings are immutable or read-only where appropriate;
- HTML methods preserve the underlying figure;
- trace indices refer to actual figure traces.

25.3 Geometry tests

- node coordinates equal prepared periodic coordinates;
- straight edge endpoints are exact;
- midpoint splitting uses exact midpoint geometry;
- segmented gradients create the requested number of intervals;
- equal aspect preserves relative spatial scale;
- reference, all-cell, and outer-boundary wireframes use the row-vector cell.

25.4 Style tests

- chemical node colors match the shared palette;
- ghost and replica style rules work;
- node outlines and alpha are preserved in marker mode;
- dot mode preserves color and alpha while using the requested compact size;
- hidden mode produces no node traces but preserves all node-key mappings;
- supported dash forms map correctly;
- unsupported markers and dashes fail;
- legend classes are deduplicated.

25.5 Hover tests

- node hover includes source key, image shift, role, and position;
- edge hover includes source edge key, endpoint images, role, and length;
- edge-hover traces do not appear as scientific graph objects;
- numeric precision is display-only.

25.6 Graph-feature tests

- directed reject and line-only modes behave correctly;
- parallel overlap policy is enforced;
- unsupported self-loops fail;
- empty graphs and isolated nodes render without crashing.

25.7 Export tests

- compact CDN HTML is written;
- self-contained HTML is written when requested;
- generated HTML contains Plotly figure data and no missing local resource path.

25.8 Na-LTA acceptance tests

Generate and inspect at least:

1. canonical-cell framework graph;
2. local-unwrapped neighborhood around a selected Si atom;
3. expanded `2 x 2 x 1` framework graph;
4. framework plus illustrative Na-O contacts;
5. perspective and orthographic camera variants.

The HTML view must allow rotation, zoom, and hover inspection. Scientific counts must remain:

```
144 framework nodes
192 framework T-O edges
48 T atoms with degree 4
96 framework O atoms with degree 2
```

Replicas and ghosts may increase display counts but must map back exactly to those source objects.

26 Performance expectations

Plotly `Scatter3d` uses WebGL, but trace count and browser payload can dominate performance. The renderer should:

- group compatible nodes and edges into shared traces;
- avoid one trace per node or edge;
- use one grouped invisible midpoint trace for edge hover where possible;
- build coordinate lists once per trace group;
- avoid Python callbacks in the first implementation;
- record final HTML size in integration diagnostics when practical.

The first renderer should target smooth inspection of graphs with approximately thousands of display nodes and several thousand display edges under simple styling. Large expanded graphs may require explicit focus or smaller image ranges.

27 Future scientific adapters

Framework, ring, site, and cage adapters may assign real-space coordinates as follows:

```
framework vertex    retained T-atom or other framework-vertex coordinate
ring node            ring-center coordinate
site node            classified site center
cage node            cage-center coordinate
```

The 3-D renderer must not assume atom-specific fields. It consumes stable keys, positions, generic metadata, and shared style rules.

28 Implemented backend summary

`plot_decorated_graph_3d()` lazily imports Plotly, calls `prepare_periodic_graph_view()`, resolves shared graph styles, groups compatible nodes and edge primitives into `Scatter3d` traces, adds midpoint hover markers, and optionally draws triclinic cell wireframes. The result exposes stable trace-index mappings and HTML export without storing new scientific state.

29 Deferred features

The first G5 implementation defers:

- trajectory animation and frame sliders;
- click callbacks and linked selections;
- browser-side filtering widgets;
- curved parallel edges;
- 3-D arrowheads;
- ring polygons, normal arrows, and aperture meshes;
- cage surfaces;
- volume rendering;
- server-backed dashboards;
- GPU rendering outside normal Plotly WebGL;
- Kaleido static export;
- synchronized 2-D and 3-D views.

30 Acceptance checklist

The G5 implementation is accepted when:

- Plotly remains optional and lazily imported;
- periodic materialization has one normative owner in G4;
- the public options contain no scientific-domain fields;
- hover mappings preserve stable source identities;
- unit-cell geometry is defined for triclinic cells;
- style compatibility and unsupported forms are explicit;
- complexity includes segments, hover points, cells, and trace groups;
- HTML export behavior is standardized;
- atomic 3-D wrappers are assigned to the adapter module;
- Na-LTA interactive acceptance views are specified;
- future framework and ring coordinates fit without renderer redesign.